

BikeMate

Motorcycle Service Assistance System

Formal Project Documentation

Prepared by	[Group Name]
Course / Subject	[Course or Subject Name]
Instructor	[Instructor Name]
Date Submitted	[Submission Date]

Group Members

- [Member 1 Full Name]
- [Member 2 Full Name]
- [Member 3 Full Name]
- [Member 4 Full Name]
- [Member 5 Full Name]

Table Of Contents

- Introduction / Background Of The Project
- Objectives
- Scope
- System Architecture
- Technology Stack
- Database Design Summary
- System Demonstration With User Interface Description
- Backend API Summary
- Testing And Verification
- Limitations
- Conclusion
- Appendix: Source Files Included

Introduction / Background Of The Project

BikeMate is a motorcycle repair and service assistance system designed to connect customers with bike shops, mechanics, and emergency repair support. The project provides Android-first mobile applications, a backend web API, a SQL Server database, and an admin web dashboard for managing users, shops, mechanics, service requests, payments, approvals, and emergency cases.

The system addresses a practical problem experienced by motorcycle users: finding reliable repair assistance can be slow, uncertain, and difficult to track. Customers may not know which nearby shop is available, which mechanic can respond, how much a service may cost, or how the repair request is progressing. Shops and mechanics also need organized tools for receiving bookings, managing services, assigning work, and building customer trust.

BikeMate responds to these needs by centralizing booking, communication, tracking, payments, verification, emergency assistance, and administration in one platform. It is implemented as a .NET-based full-stack system using .NET MAUI for mobile applications, ASP.NET Core Web API for backend services, Entity Framework Core and SQL Server for persistence, and Blazor for the system admin web dashboard.

Objectives

General Objective

To develop BikeMate, a motorcycle service assistance platform that helps customers book motorcycle repair services, connect with mechanics and shops, request emergency help, communicate in real time, and allow administrators to monitor and manage platform activity.

Specific Objectives

- Provide a mobile app where customers can register, log in, manage their profile, add motorcycle details, browse shops and services, and create service requests.
- Allow customers to schedule repairs, upload service details, choose service providers, track service status, view payment records, and submit reviews.
- Provide an emergency assistance flow where customers can request urgent help, share location, communicate with responders, and track active emergency requests.
- Provide a mechanic/rider interface where mechanics can view assigned jobs, accept or reject work, update repair progress, manage profile details, and view earnings and ratings.
- Provide a shop admin app where shop owners can register, submit business information, manage shop profile, products, services, bookings, mechanics, messages, notifications, reports, and settings.
- Provide a system admin dashboard for user management, shop verification, mechanic verification, service request monitoring, payment monitoring, emergency queue review, and admin account management.
- Store platform data in a structured SQL Server database with entities for users, roles, shops, mechanics, clients, motorcycles, services, products, bookings, messages, payments, reviews, notifications, live locations, and audit logs.
- Implement secure authentication using JWT, password hashing, OTP support, and role-based authorization.

- Support real-time behavior through SignalR hubs for booking, chat, emergency, location, and notification features.

Scope

The scope of BikeMate includes the major workflows needed for a local demonstration of a motorcycle service marketplace. The system covers customer service booking, mechanic job handling, shop management, backend API processing, database persistence, and system administration.

- User account registration, login, OTP verification, Google login structure, password reset, logout, and role-based routing.
- Customer profile management, address management, motorcycle records, service booking, scheduling, request tracking, payments, receipts, invoices, chat, notifications, profile, and emergency assistance.
- Mechanic dashboard, job list, active job details, map/location support, emergency requests, chat, profile editing, earnings, ratings, and service history.
- Shop admin registration, shop application submission, business permit and valid ID workflow, OTP verification, shop profile, products, services, bookings, mechanics, messages, sales, reports, notifications, settings, and help support.
- System admin dashboard, users list, shop records, approvals, mechanics directory, service requests, emergency requests, payments, admin account management, and detailed review pages.
- Backend endpoints for authentication, customers, mechanics, shops, shop onboarding, services, products, service requests, payments, conversations/messages, locations, emergency, maps/geography, notifications, devices, files, reports, and admin functions.

Project Boundary

The current project is local/development-ready. It includes real project structure, database schema, API implementation, mobile screens, web admin screens, tests, screenshots, and local build artifacts. Production use would require real third-party keys for payment, OAuth, email, notifications, and storage.

System Architecture

Project	Purpose
BikeMate.Mobile	Android-first customer, mechanic, and system admin mobile app screens.
BIKEMATES_ADMIN	.NET MAUI shop admin mobile app for shop owner workflows.
BikeMate.Api	ASP.NET Core backend API for authentication, service requests, messaging, payments, location, emergency, and admin endpoints.
BikeMate.Core	Shared DTOs, constants, and entity definitions used by the API and mobile clients.
BikeMate.Infrastructure	Entity Framework Core database context, migrations, and SQL scripts.
BikeMate.WebAdmin	Blazor system admin dashboard.
BikeMate.Tests	Unit tests for services, middleware, helpers, and constants.

Technology Stack

Layer	Technology
Mobile Application	.NET MAUI, C#, XAML, Android target
Shop Admin App	.NET MAUI, C#, XAML
Backend API	ASP.NET Core Web API, JWT bearer authentication
Web Admin	Blazor, Razor components, Bootstrap styling
Database	SQL Server / SQL Server Express
Data Access	Entity Framework Core migrations and DbContext
Real-Time Features	SignalR hubs for booking, chat, emergency, location, and notifications
Testing	xUnit test project
Optional Integrations	PayMongo, Google OAuth, SMTP/SendGrid, Firebase, file storage

Database Design Summary

The database stores both operational records and governance records. It supports identity management, service marketplace data, repair transactions, communication, payment records, verification, and auditability.

- Identity: Users, Roles, UserRoles, AuthProviders, OTP records, password reset tokens, and device tokens.
- Customer records: Clients, client addresses, and motorcycles.
- Mechanic records: Mechanics, availability, assigned shop mechanics, ratings, completed job counts, and live locations.
- Shop records: Shops, operating hours, services, service categories, products, product images, and service images.
- Service transactions: Service requests, request status, request status history, request media, live tracking, payments, and reviews.
- Communication: Conversations, conversation participants, messages, notifications, and emergency status updates.
- Governance: Audit logs, account status, shop status, verification data, and approval history.

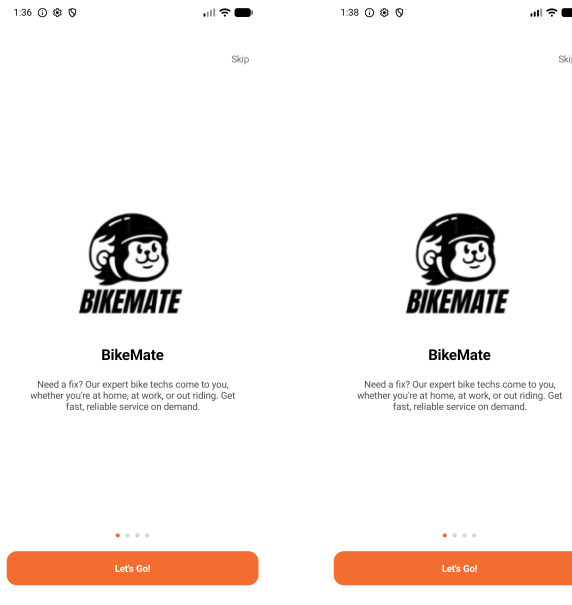
Database-related files included in the project:

- BikeMate.Infrastructure/Migrations
- BikeMate.Infrastructure/Scripts/BikeMate_InitialSchema.sql
- BikeMate.Infrastructure/Scripts/BikeMate_RunThis_DatabaseSetup.sql
- BikeMate.Infrastructure/Scripts/sql-server-triggers.sql
- tools/Reset-DevData.sql
- tools/Prepare-DemoDatabase.ps1

System Demonstration With User Interface Description

1. Onboarding And Login

The system starts with BikeMate onboarding and login screens. After authentication, JWT tokens are stored and the user is routed to the correct role-based shell. The login flow supports customer, mechanic, shop admin, and system admin accounts.

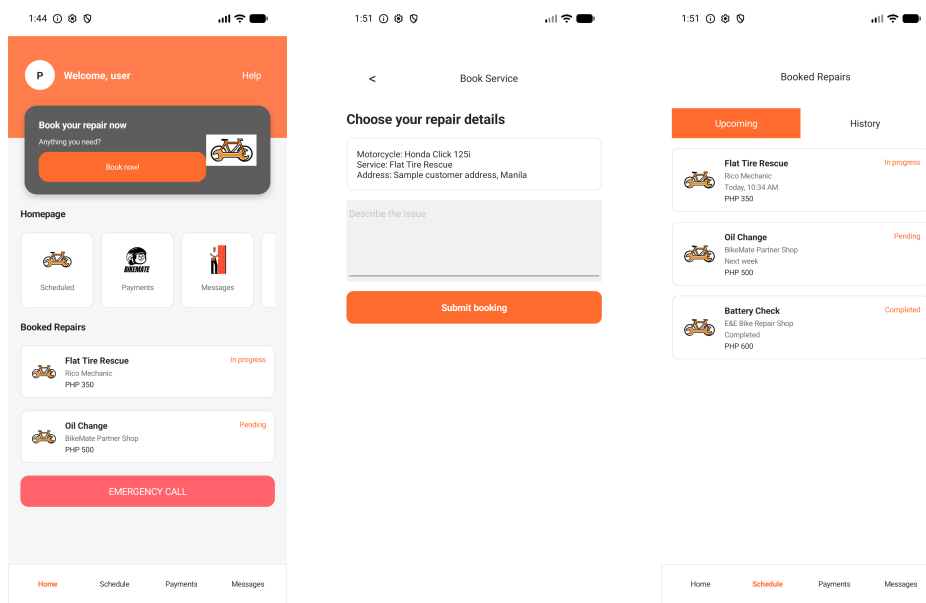


Login screen

Onboarding screen

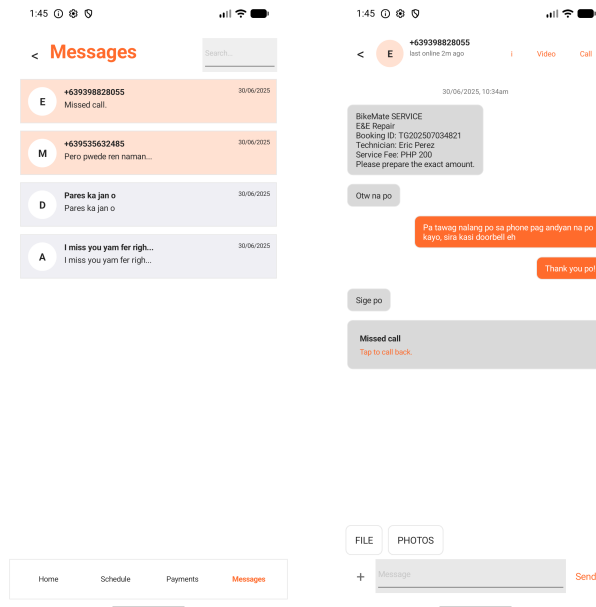
2. Customer Home And Booking

The customer interface allows users to access home, schedule, messages, payments, profile, help, and booking functions. During booking, the customer can describe the issue, choose a service type, set a schedule, upload details, confirm the request, and proceed to tracking and payment.



3. Messages And Chat

BikeMate includes messaging screens so customers, mechanics, and service participants can coordinate details. The backend includes conversation and message endpoints, and SignalR support is prepared for real-time communication.

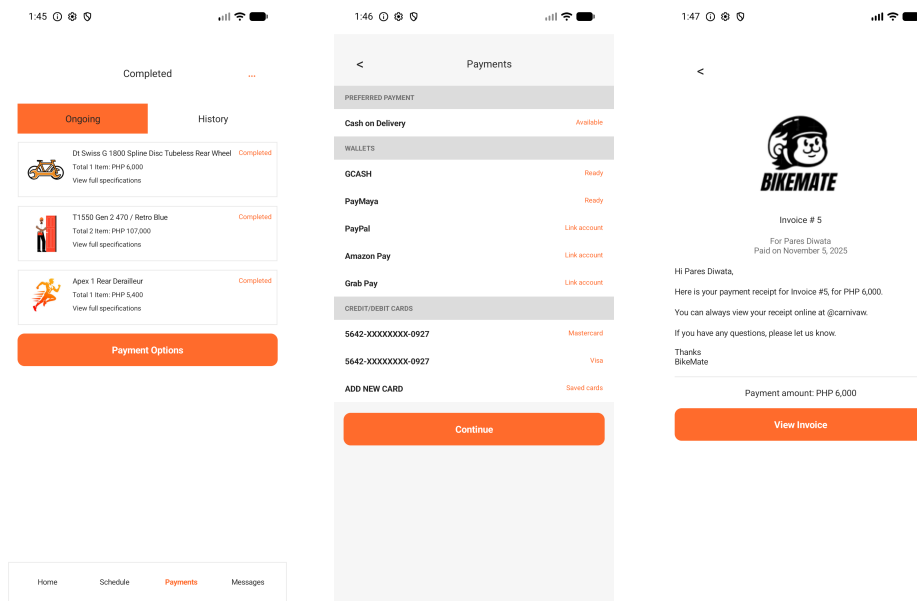


Messages list

Chat conversation

4. Payments, Receipt, And Invoice

Payment screens show payment history, options, payment details, receipts, and invoices. The backend includes a PayMongo checkout structure, with real transactions requiring valid production keys.



Payment history

Payment options

Receipt

5. Emergency Assistance

Emergency assistance allows a customer to send an urgent repair request with location details. The API can identify nearby responders, create emergency request records, support emergency conversation, and manage call/session state. Admin and mechanic screens can monitor and respond to emergency requests.

- Emergency SOS page.
- Emergency location picker.
- Calling emergency page.
- Emergency live call page.
- Active emergency tracking page.
- Admin emergency queue in the web dashboard.
- Mechanic emergency request screen.

6. Mechanic Workflow

Mechanics can view their dashboard, jobs, active job details, map and location support, emergency requests, chat, profile editing, earnings, ratings, and service history. The mechanic workflow supports accepting or rejecting work and updating service status as the job progresses.

7. Shop Admin Workflow

The shop admin app supports owner registration, account creation steps, document submission, OTP verification, shop application review, shop profile management, products, services, bookings, mechanics, messages, sales, reports, notifications, settings, and help support.

8. System Admin Web Dashboard

The Blazor web dashboard gives system administrators platform-wide monitoring and control. Admin users can review the dashboard, emergency queue, users, shops, approvals, mechanics, repair requests, payments, and admin accounts.

Backend API Summary

The backend API exposes the main business workflows through authenticated and role-aware endpoint groups. It also maps SignalR hubs for real-time features.

Area	Examples
Authentication	Register, login, Google login structure, OTP verification, password reset, logout, current user.
Customer	Profile, address, motorcycles, dashboard, home status.
Mechanic / Rider	Profile, nearby mechanics, jobs, active jobs, accept/reject, status update, photos, location, earnings, ratings.
Shop	Dashboard, profile, application, services, bookings, mechanics, inventory, payments, reviews, analytics.

Service Requests	Create request, active/my requests, status updates, cancellation, mechanic assignment, media upload, shop selection, timeline, upcoming, history.
Emergency	Create emergency request, status, conversation, cancel, accept, call start/end, nearby responders.
Administration	Dashboard, users, customers, mechanics, shops, pending approvals, requests, emergency requests, payments, reports, audit logs, announcements.
Real-Time Hubs	/hubs/booking, /hubs/chat, /hubs/emergency, /hubs/location, /hubs/notification.

Testing And Verification

The final project test report shows successful verification of the core local demonstration workflow.

- Database migration and seed lookup data verified.
- API build, run, and health endpoint verified.
- Login passed for Customer, Mechanic, Shop Admin, and System Admin.
- Role-based authorization passed.
- Customer, Mechanic, Shop Admin, and System Admin API data verified.
- Chat REST endpoints and payment checkout placeholder verified.
- Android project build and emulator launch verified.
- Onboarding images and mobile packaging verified.

Role	Email	Password
Customer	customer@bikemate.test	Password123!
Mechanic	mechanic@bikemate.test	Password123!
Shop Admin	shop@bikemate.test	Password123!
System Admin	admin@bikemate.test	Password123!

Limitations

- Real PayMongo payment processing requires valid PayMongo public, secret, and webhook keys.
- Real Google OAuth requires a valid Google OAuth client ID.
- Real email/OTP delivery requires SMTP or SendGrid credentials.
- Real push notifications require Firebase credentials.
- Production file storage requires a configured local or cloud storage provider.
- The project is primarily configured for local development, classroom demonstration, and Android-first testing.

Conclusion

BikeMate successfully demonstrates a full-stack motorcycle service platform with mobile, backend, database, and web admin components. The project supports the major workflows needed by customers, mechanics, shop owners, and system administrators. Its main contribution is the organization of motorcycle service assistance into a single system that supports booking, tracking, messaging, payment records, verification, emergency response, and administrative control.

Although production deployment still requires real third-party keys and cloud configuration, the current implementation is ready for local demonstration and provides a strong foundation for a real motorcycle service marketplace.

Appendix: Source Files Included

Requirement	Included Files / Folders
Mobile App Code	BikeMate.Mobile, BIKEMATES_ADMIN
Web Service / API Code	BikeMate.Api, BikeMate.Core, BikeMate.Infrastructure
Database Schema And Associated Files	BikeMate.Infrastructure/Migrations, BikeMate.Infrastructure/Scripts, tools/Reset-DevData.sql, tools/Prepare-DemoDatabase.ps1
Web App Files	BikeMate.WebAdmin
Tests And Supporting Docs	BikeMate.Tests, README.md, setup guides, API testing guide, final test report, inspection report
Presentation / Demo Assets	BikeMate screenshots and generated PowerPoint deck
Build Artifacts If Required	artifacts/phone-demo/apk/BikeMate.Mobile.Debug.apk, artifacts/phone-demo/apk/BIKEMATES_ADMIN.Debug.apk